CSL301: Operating System

Segmentation Lab

Module 1: Downward-Growing Stack Translation

Task: Implement address translation for downward-growing segments (Stack) in C. Calculate Physical Address = Base - Offset when segment direction is downward. Ensure bound check (Offset < Limit) triggers Segmentation Fault on invalid access.

Segment ID	Segment Type	Base Address	Limit	Query Offset	Expected Output
Seg 2	Stack (Downward)	4700	400	100	**Physical Address: 4600**
Seg 2	Stack (Downward)	4700	400	500	**SEGMENTATION FAULT**

Module 2: Multi-Ring Hardware Privilege Checker

Task: Extend segment structure with min_ring (Ring 0: Kernel to Ring 3: User). Deny access with a PRIVILEGE VIOLATION TRAP if current_cpu_ring > segment.min_ring, even if bitwise permissions (R/W/X) pass.

Process Ring	Target Segment	Segment Min Ring	Req. Access	Allowed Perms	Expected Result
Ring 3 (User)	Seg 0 (Kernel Code)	Ring 0	READ (1)	READ | EXEC (5)	**TRAP: Privilege Violation**
Ring 0 (Kernel)	Seg 0 (Kernel Code)	Ring 0	READ (1)	READ | EXEC (5)	**SUCCESS**

Module 3: Best-Fit Allocation with Block Splitting

Task: Implement the best_fit_allocate(int req_size) function. Find the smallest free hole that satisfies req_size. Allocate the segment and split the leftover space into a new free block hole.

Available Free Holes	Requested Size	Chosen Hole Base	New Leftover Hole
Hole A: Base 200 (Size 300) Hole B: Base 700 (Size 200)	150 Units	**Base 700 (Size 200)**	**Base 850 (Size 50)**

Module 4: Memory Compaction & Byte Migration

Task: Implement compact_memory() using C's memmove(). Shift active segment payloads in simulated char RAM[1000] to low memory contiguous blocks, update segment base addresses, and clear old positions with memset().

Segment	Initial Base	Payload String	Compacted Base	Updated Memory Offset
Seg 0	Base 200	"HELLO"	**Base 0**	**RAM[0..4] = "HELLO"**
Seg 1	Base 600	"WORLD"	**Base 6**	**RAM[6..10] = "WORLD"**